

Trabajo Integrador

Parte 1 y 2

Grupo 16

Lenguajes: Python y JavaScript.

Integrantes:

- Colli, Lucio 18394/7
- Fierro, Santiago 21875/5
- Grillo Estiu, Ramiro Nicolas 21115/9
- Santa Cruz, Joaquin Emanuel 22432/4

Bibliografía:

- <https://www.python.org/doc/>
- <https://doc.pypy.org/en/latest/>
- <https://cs106a.stanford.edu/textbook>
- <https://cs.stanford.edu/people/nick/py/>
- <https://developer.mozilla.org/en-US/docs/Web/JavaScript>
- <https://devdocs.io/javascript/>
- <https://web.stanford.edu/class/cs98si/slides/overview.html>
- <https://web.stanford.edu/class/cs98si/slides/data-and-variables.html>
- *JavaScript: The Good Parts* by Douglas Crockford
- *JavaScript: The Definitive Guide* by David Flanagan

Introducción

Es fundamental estudiar los distintos aspectos de los lenguajes de programación para así tener un mayor entendimiento de estos y, a su vez, desarrollar habilidades que nos serán útiles en el futuro. El estudio de los lenguajes de programación nos permite tener una mirada más crítica sobre ellos, permitiéndonos ser capaces de elegir el apropiado para un momento determinado, poder aprender características desconocidas de un lenguaje conocido y poder aprender los nuevos lenguajes que vayan surgiendo de manera más fácil. Por ello, resulta esencial la comparación entre los lenguajes que a continuación se exponen, debido a que son dos muy populares y poderosos, pero que presentan ciertas diferencias que trataremos en este documento.

Los temas a tratar en el documento son los siguientes: criterios de evaluación de los lenguajes, estructuras de control, aspectos semánticos, tipos de variables, parámetros, sistema de tipos y manejo de excepciones.

Simplicidad y Legibilidad

Cuando hablamos de que un lenguaje sea simple y legible nos referimos a la forma que tiene este a la hora de producir programas. El grado de facilidad que brinda para escribirlos, leerlos, aprenderlos o enseñarlos. Tanto JavaScript como Python son lenguajes de programación reconocidos por su simplicidad y flexibilidad. Ambos adoptan un enfoque de tipado dinámico, lo que implica que no es necesario especificar el tipo de variable al declararlas. Esta característica facilita la escritura y lectura del código al eliminar la necesidad de definir tipos de datos explícitamente.

Además, tanto JavaScript como Python presentan una sintaxis limpia y clara, lo que contribuye a su facilidad de uso. Por ejemplo, en JavaScript, no es obligatorio finalizar cada instrucción con un punto y coma (;), lo que hace que el código sea más legible y menos propenso a errores de sintaxis.

Ambos lenguajes también proporcionan una amplia gama de métodos para manipular listas y otros tipos de estructuras de datos, con nombres intuitivos que facilitan la comprensión del código y lo hacen más conciso.

```
// Cálculo del promedio de una lista de números en JavaScript
// Variables a ingresar en la lista
let a = 10;
let b = 5;
let c = 25.1;

// Lista
const numbers = [a,b,c];

// Cálculo del promedio
const average = numbers.reduce((acc, curr) => acc + curr, 0) / numbers.length;

console.log("El promedio es:", average);
```

JavaScript

```
# Cálculo del promedio de una lista de números en Python

# Variables de la lista
a = 10
b = 5
c = 25.1

# Lista de números
numbers = [a,b,c]

# Cálculo del promedio
average = sum(numbers) / len(numbers)

print("El promedio es:", average)
```

Python

Confiabilidad

Nos referimos a que un lenguaje es confiable cuando nos da la seguridad de no dejar pasar los errores de tipo y por brindar manejo de excepciones, para poder atender errores en tiempo de ejecución.

Una desventaja significativa de la flexibilidad del tipado dinámico que se mencionó anteriormente es su impacto en la confiabilidad del código. La capacidad de las variables para cambiar de tipo durante la ejecución del programa puede conducir a errores difíciles de detectar, lo que a su vez puede afectar la confiabilidad y la estabilidad del software.

Por otro lado, ambos cuentan con manejo de excepciones, lo que permite atender errores en tiempo de ejecución, pero con una importante diferencia en cuanto al tipado de cada uno de ellos. Python es un lenguaje fuertemente tipado, es decir, al crear un objeto se vincula para siempre su conjunto de operaciones que son válidas para su tipo. JavaScript se caracteriza por estructuras de tipado débil, donde la declaración de variables no exige la asociación con un tipo de datos de forma implícita y unívoca. Si bien el lenguaje los asigna por nosotros de forma interna, la característica de tipado débil podría considerarse un foco potencial de errores. Como ejemplo, podemos ver el siguiente fragmento de código en donde se utiliza el manejo de excepciones del que se habló anteriormente:

```
let str = "Hola";
let num = 5;
try {
  let result = str + num;
  console.log("Resultado:", result);
} catch (error) {
  console.error("Error:", error.message);
}
```

Este segmento de código puede ejecutarse sin ningún problema, aunque esto podría tratarse de un error y jamás nos daríamos cuenta, dado que no lo detecta como tal.

JavaScript

```
str_value = "Hola"
num_value = 5

try:
    result = str_value + num_value
    print("Resultado:", result)
except Exception as e:
    print("Error:", e)
```

En cambio, este segmento de código generaría un error en tiempo de ejecución y lo imprimiría en la terminal, ya que Python no permite realizar transformaciones implícitas a los tipos de variables.

Python

Estructuras de control

En cuanto a las estructuras de control **for** e **if**, ambos lenguajes comparten conceptos similares pero difieren en su sintaxis y algunas características específicas.

En los siguientes ejemplos, ambos fragmentos de código presentan un bucle "for" que itera hasta que la variable "i" alcanza el valor de 5. Dentro de este bucle, se incluye una estructura "if" que evalúa si "i" es par o impar, imprimiendo el valor correspondiente en cada caso. Las discrepancias sintácticas entre los dos lenguajes son notables: en JavaScript, el cuerpo de la estructura está encerrado entre llaves ("{" y "}"), mientras que en Python se emplea el ":" y todas las instrucciones indentadas pertenecen a dicha estructura.

En Python y JavaScript se utiliza la palabra clave 'for'. En el primero, le sigue un identificador con alcance solo de bloque, luego la palabra clave 'in' seguido de un iterable, finalizando con ":" por ser una instrucción compuesta. En JavaScript, le siguen tres declaraciones entre paréntesis, separadas por ";": se define una

variable que controlará el bucle, luego una expresión condicional de corte y luego una operación a realizar luego de ejecutar cada iteración (generalmente, incrementar la variable que controla el bucle), cabe destacar que las tres partes del for son opcionales, ya que se puede realizar previamente la inicialización de la variable i, omitir la condición de corte si se evalúa dentro del bloque y se utiliza la sentencia break y el incremento de la variable índice si se incrementa dentro del bucle. Esto puede provocar errores difíciles de detectar, e incluso bucles infinitos.

Es importante tener en cuenta que la variable 'i' del for puede ser modificada y generar una ejecución inesperada o no deseada del bucle, siempre y cuando esto ocurra dentro de la estructura del 'for'. En el caso de que se intente modificar la variable índice en una función externa que se llame dentro de la ejecución del bucle, esto no tendrá efecto y no causará ningún problema.

```
for i in range(5):
    if i % 2 == 0:
        print("El numero es par")
    else:
        print("El numero es impar")
```

Python

```
for (let i = 0; i < 5; i++) {
    if (i % 2 == 0){ // Se imprime que el numero es par
        console.log("Par")
    }
    else{ // Se imprime que el numero es impar
        console.log("Impar")
    }
}
```

JavaScript

En JavaScript hay varias formas de escribir los bucles "for". En el ejemplo se utilizó la forma básica, indicando la variable a utilizar, condición de salida y la expresión de actualización. Otras variaciones del "for" serían las siguientes:

```
const array = ["a", "b", "c", "d", "e"];

console.log("Usando bucle 'for...of':");
for (const elemento of array) {
    console.log("Elemento:", elemento);
}
```

- Bucle for...of, usado generalmente para recorrer vectores

```
const array = ["a", "b", "c", "d", "e"];

console.log("Usando forEach()");
array.forEach(function(elemento) {
    console.log("Elemento:", elemento);
});
```

- Método forEach(), en la cual a cada elemento se le realiza una acción, evitando el uso de bucles.

```

1  async function obtenerDatos() {
2      const datos = [1, 2, 3];
3      for await (let dato of datos) {
4          console.log(dato);
5      }
6  }
7
8  obtenerDatos();

```

- Este bucle se utiliza para iterar sobre iterables asíncronos, como Promises o Streams, esperando la resolución de cada uno antes de continuar con la siguiente iteración.

En Python, también existe otra estructura para realizar el for:

```

array = [1,2,3,4]

for element in array:
    print(element)

```

En donde “element” es una variable que por cada iteración toma el valor del elemento actual de “array”, que puede ser cualquier objeto en el que se puede iterar, ya sea arreglo, tupla, lista, cadena de caracteres, etc. En este caso es una lista. JavaScript también cuenta con esta estructura.

Aspectos Semánticos

Python y JavaScript hacen un breve análisis de semántica estática, cuando el programa se carga en memoria, pero es principalmente semántica dinámica. Al ser lenguajes interpretados, esto significa que tienen un intérprete que traduce las instrucciones, una por una a lenguaje máquina para que se ejecuten al momento. Por un lado, esto trae la ventaja de que no debe compilar todo el código antes de ejecutarse, incluso las partes que jamás va a ejecutar (por el flujo del programa); no obstante, esto puede llevar a que haya líneas de código que, al no ejecutarse, podrían contener errores sintácticos y/o semánticos que no detectaremos incluso tras ejecutar el programa.

Algunos IDE, como PyCharm para Python y Visual Studio Code para JavaScript, pueden resaltar errores y advertencias en el editor, entre otras cosas.

Además, existen diversos intérpretes para cada lenguaje, para Python, uno de ellos es PyPy, que puede hacer compilación JIT (Just In-Time), en la cual analiza y compila ciertas partes de código en lenguaje de bajo nivel en tiempo de ejecución. Esta compilación conlleva un pequeño análisis de semántica estática, pero no es muy profundo, ya que la mayoría se hace dinámicamente. Un ejemplo de intérprete en JavaScript que use compilación JIT podría ser el motor V8, desarrollado por google, y usado en NodeJs.

En este ejemplo de Python obtendremos un error previo a la ejecución, ya que en Python no se admiten operaciones entre distintos tipos, lo cual es detectado por la semántica estática. En este caso se está queriendo sumar un valor de tipo int con un string.

```

1  numero1 = 5
2  numero2 = "5"
3
4  resultado = numero1 + numero2
5  print(resultado)

```

Python

Un ejemplo de semántica dinámica puede ser el siguiente, ya que se detecta en el momento de la ejecución.

```
1 num1 = 10
2 num2 = 0
3 resultado = num1 / num2
4 print(resultado)
```

Python

Mismo caso que el ejemplo anterior, pero podríamos no detectar el error ya que JavaScript admite este tipo de operaciones.

```
1 let numero1 = 5
2 let numero2 = "5"
3
4 resultado = numero1 + numero2
5 console.log(resultado)
```

JavaScript

Tipos de Variables.

En Python, el alcance de las variables depende del contexto en el que fueron declaradas. En el segmento de código proporcionado podemos ver que la variable “numero1” es global a todo el programa, ya que se encuentra dentro del cuerpo del programa principal, mientras que las variables “d” y “c” son locales a la función “suma”. Una característica interesante es que se puede utilizar la palabra reservada **global**, que permite indicar que se utilizara de manera local una variable declarada fuera de la función. En cambio, en JavaScript no es igual, ya que por ejemplo la variable “numero1” de la foto derecha tiene un alcance global ya que fue declarada con la palabra reservada **var**, mientras que la variable “aux”, que se encuentra dentro de la función, tiene un alcance de bloque debido a su declaración con la palabra reservada **let**. No obstante, no todas las variables en JavaScript son dinámicas, si se declara una variable con la palabra reservada **const**, se considerará estática ya que el tipo con el que se declare no va a poder cambiar en la ejecución del código.

Python

```
1 numero1 = 5
2
3 def suma(a, b):
4     d = 2
5     global numero1
6     c = numero1
7     return a + b + c + numero1 + d
8
9 print(suma(1, 4)) ##imprime 17
```

JavaScript

```
1 var numero1 = 5
2
3 function suma(a, b){
4     let aux = 1
5     return a + b + aux
6 }
7
8 console.log(suma(numero1, 4)) // imprime 10
```

Python

Línea	Identificador	L-Value	R-Value	Alcance	Tiempo de Vida
1	numero1	Dinámica	Basura	2-9	1-9
3	suma			4-9	3-7
3	a	Dinámica	Basura	4-7	3-7
3	b	Dinámica	Basura	4-7	3-7
4	d	Dinámica	Basura	5-7	4-7
6	c	Dinámica	Basura	7	6-7

JavaScript

Línea	Identificador	L-Value	R-Value	Alcance	Tiempo de Vida
1	numero1	Automática	Basura	2-8	1-8
3	suma			4-8	3-6
3	a	Automática	Basura	4-6	3-6
3	b	Automática	Basura	4-6	3-6
4	aux	Automática	Basura	5-6	3-6

Parámetros

Un parámetro es un valor que la función espera recibir cuando sea llamada invocada, a fin de ejecutar acciones en base al mismo. Una función puede esperar uno o más parámetros (que irán separados por una coma) o ninguno.

Los parámetros, tanto en Python como en JavaScript se indican entre los paréntesis, a modo de variables, a fin de poder utilizarlos como tales, dentro de la misma función.

Ahora bien, la ligadura entre el parámetro real y formal es según el orden en que se posicionan, y en principio esto es así en ambos lenguajes, pero ahora veremos que en Python puede no serlo siempre.

Python:

En este ejemplo se pueden ver 4 tipos de parámetros diferentes, 'a' que es posicional y se debe pasar siempre que se invoque a la función, 'b' que en el caso de que no se le proporcione un valor, tomará el valor por defecto que se encuentra en la declaración de la función, '*args' que va a tomar todos los valores que sean pasados después de los dos primeros parámetros y los almacenará en una tupla, '**kwargs', que es muy similar al anterior, pero almacena en un diccionario aquellos parámetros que se hayan pasado con un nombre y un valor, y finalmente tenemos el parámetro 'c' que se comporta como un parámetro nombrado lo que quiere decir que en la invocación de la función podemos asignarle un valor a través del nombre del parámetro formal.

```
1 def example_function(a, c, b=2, *args, **kwargs) -> None:
2     print("a:", a) # Positional or keyword
3     print("b:", b) # Positional or keyword with default
4     print("args:", args) # Variable positional arguments
5     print("kwargs:", kwargs) # Variable keyword arguments
6
7 # Llamadas a la funcion
8 example_function(1)
9 example_function(1, 3)
10 example_function(1, 3, c=10)
11 example_function(1, 3, 4, 5, c=10)
12 example_function(1, 3, 4, 5, x=6, y=7, c=10)
13
```

Javascript:

En este ejemplo se pueden ver 3 tipos de parámetros diferentes, los dos primeros son igual a los mencionados anteriormente para el ejemplo de Python, 'a' es posicional obligatorio cada vez que se llama a la función y 'b' va a tomar el valor por defecto declarado en el parámetro formal de la función en el caso de que no se le proporcione un valor, finalmente '...args' es muy similar al '*args' de Python, lo que hace es almacenar en un array el resto de los valores pasados por parámetro en la invocación de la función. Javascript no cuenta con parámetros nombrados, pero se podría simular este tipo de parámetros utilizando objetos.

```
1 function exampleFunction(a, b = 2, ...args) {
2     console.log("a:", a); // Positional
3     console.log("b:", b); // Positional with default
4     console.log("args:", args); // Rest parameters
5 }
6
7 // Llamadas a la funcion
8 exampleFunction(1);
9 exampleFunction(1, 3);
10 exampleFunction(1, 3, 4, 5);
11
```

En cuanto al tipo de pasaje de parámetros en Python, se pueden pasar por valor o por referencia. Los tipos de datos simples (inmutables) como enteros, booleanos y cadenas de caracteres se pasan por valor, mientras que los tipos de datos compuestos o mutables se pasan por referencia.

En JavaScript, los argumentos siempre se pasan por valor. Cuando una variable hace referencia a un objeto, el "valor" es la referencia misma. Mutaciones dentro de la función no afectan el valor fuera de la misma.

Sistema de Tipos

Un sistema de tipos es un conjunto de reglas que el lenguaje utiliza para estructurar y organizar sus tipos. El objetivo del sistema de tipos es escribir los programas más seguros posibles. Los errores de tipo pueden chequearse de dos maneras o en dos momentos: de manera estática o de manera dinámica.

Para comenzar el análisis de los sistemas de tipos en Python y JavaScript, vamos a notar que ambos son de tipado dinámico (esto quiere decir que la ligadura se produce en tiempo de ejecución), pero la diferencia que encontramos es que Python es Fuertemente tipado (impone restricciones que aseguran que no se producirán errores de tipo en ejecución) y JavaScript no.

Veamos unos ejemplos simples de porque tanto JavaScript como Python son tipado dinámico. Podemos asegurar esto debido a que las siguientes operaciones no producen error:

Python	JavaScript
<pre>1 num = 1 2 num = 'a'</pre>	<pre>1 let num = 5; 2 num = 4.5; 3 num = 'a';</pre>

Justamente vemos cómo en ambos casos a la variable num se le puede asignar cualquier tipo, y esto sucede ya que la variable no tiene un tipo, y puede nombrar cualquier objeto. Después, una vez asignado un valor, si queremos saber el tipo del valor asignado bastará con usar la función Type() en Python o typeof en JavaScript para que nos indique a qué tipo pertenece.

Continuemos viendo unas diferencias entre ambos lenguajes con respecto al tipado de ambos, dijimos que Python es fuertemente tipado y JavaScript es débil, veamos por qué afirmamos esto.

Como dijimos previamente un lenguaje se dice que es fuertemente tipado si el sistema de tipos impone restricciones que aseguran que no se producirán errores de tipo en ejecución, una forma simple de ejemplificar esto en Python seria mostrando que no podemos combinar tipos:

Python:

```
1 num = 1
2 car = 'a'
3 print(num + car)
```

```
Traceback (most recent call last):
  File "/home/santiago/Documentos/Tercero/Ing 2/ProyectoFierro", line 3, in <module>
    print(num + car)
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

Esto no se nos permitirá en Python, para realizar esta operación, hay que hacer alguna conversión explícita del dato en conflicto en cuestión (por ejemplo: int(car)), en cambio en JavaScript si.

Javascript:

```
1 let num1 = 5;
2 let num2 = 'a'
3 console.log(num1 + num2) >> '5a'
```

Como vemos el resultado de la suma es '5a' y vemos que se permite, pero ahora veamos un poco de porque esa operación se permite.

Javascript presenta una peculiaridad con respecto a los operadores "-" y "+". Podemos usar el operador "+" para realizar las operaciones de suma normales, e incluso para realizar operaciones de concatenación entre un string y un entero.

Ahora bien, Python a pesar de ser un lenguaje de tipado dinámico, soporta opcionalmente un sistema de tipos estático. Usando anotaciones, podemos indicar el tipo de dato de una variable al crearla:

```
1 num: int = 5
2 print (num) >> 5
```

En JavaScript, el lenguaje base no soporta tipado estático. Sin embargo, TypeScript, que es un superconjunto de JavaScript, introduce un sistema de tipos estático opcional, similar a las anotaciones de tipo en Python.

Manejo de excepciones

En este punto hablaremos del manejo de las excepciones en los lenguajes de Python y JavaScript, pero antes definamos que es una excepción. Una excepción es una situación anómala, o inesperada, que se da en la ejecución de un programa y que se supone que ocurre con poca frecuencia, y la cual no podemos manejar.

A la hora de hablar de excepciones, podemos encontrar varios puntos importantes. En este trabajo hablaremos sobre su definición, la forma de invocarlas, cómo manejarlas y un criterio de continuación.

Para este tipo de problemas existe la técnica del manejo de excepciones, la cual nos permite justamente manejar o controlar estos errores en ejecución.

Python:

Comenzando con Python, para el manejo de excepciones, este nos brinda ciertas palabras reservadas las cuales nos permiten manejar las excepciones que puedan surgir, en el caso el manejo de excepciones se hace mediante los bloques que utilizan las sentencias try, except, else y finally.

En este lenguaje, ya existen excepciones predefinidas (built-in exceptions). Sin embargo, podemos definir nuevas excepciones extendiendo la clase base Exception con una excepción personalizada que contenga las características deseadas por el programador. Por otro lado, en cuanto a la manera de invocarlas, se puede realizar mediante la instrucción 'raise' seguido del nombre de la excepción, junto a un mensaje opcional, y se pueden manejar mediante la sentencia 'except' y su nombre. Finalmente, el criterio de continuación que tiene Python es por 'Terminación', lo que quiere decir que una vez levantada la excepción en un bloque try, esta es manejada (en caso de que exista un manejador para la misma) y dicho bloque se da por finalizado.

Un ejemplo puede ser el siguiente:

```
1 class MiExcepcion(Exception):
2     """Una excepción personalizada simple"""
3     pass
4
5 # Uso de la excepción personalizada
6 try:
7     raise MiExcepcion("Ocurrió un error personalizado")
8 except MiExcepcion as e:
9     print(f"Capturada: {e}")
```

Python

try: El bloque de código donde puede ocurrir una excepción.

except: Como mencionamos anteriormente, el 'except' captura y maneja las excepciones específicas. Puede tener múltiples bloques 'except' para diferentes tipos de excepciones.

else: Se ejecuta si no ocurre ninguna excepción en el bloque try.

finally: Se ejecuta siempre, ocurra o no una excepción, ideal para la limpieza de recursos.

```
1  def divide(a, b) -> Any | None:
2      try:
3          result = a / b
4      except ZeroDivisionError as e:
5          print(f"Error: {e}")
6          return None
7      except TypeError as e:
8          print(f"Error: {e}")
9          return None
10     else:
11         print("Division satisfactoria.")
12         return result
13     finally:
14         print("Ejecucion completada.")
15
16 print(divide(10, 2))
17 print(divide(10, 0))
18 print(divide(10, "a"))
```

En el primer caso se ejecuta el bloque try con los valores a=10 y b=2, la ejecución de la división no levanta ninguna excepción, por lo que se ejecuta el bloque else imprimiendo "División satisfactoria", retornando el resultado y se ejecuta el bloque finally imprimiendo 'Ejecución completada'.

En el segundo caso se ejecuta el bloque try con los valores a=10 y b=0, la ejecución de la división genera un error de tipo ZeroDivisionError que es capturada en el primer except, imprimiendo "Error: ZeroDivisionError", por último se ejecuta el finally imprimiendo 'Ejecución completada'.

En el tercer caso se ejecuta el bloque try con los valores a=10 y b='a', la ejecución de la división levantará un error de tipo que será capturada por el 'except TypeError', imprimiendo "Error: TypeError" y por último se ejecuta el finally imprimiendo "Ejecución completada".

JavaScript:

El manejo de excepciones en JavaScript consiste en el uso de las cláusulas try...catch y finally, o bien lanzarlas a través de la cláusula throw.

Al igual que en Python, en JavaScript se pueden definir excepciones personalizadas creando una clase que herede de la clase 'Error' con las características deseadas. Luego, para invocar una excepción se utiliza la sentencia 'throw new' seguida del nombre de la excepción, la cual puede manejarse a través de la palabra clave 'catch' y la excepción a manejar entre paréntesis. Por último, el criterio de continuación que utiliza este lenguaje es por Terminación.

Un ejemplo puede ser el siguiente:

```
1 class MiErrorPersonalizado extends Error {
2   constructor(mensaje) {
3     super(mensaje); // Llama al constructor de la clase Error
4     this.name = 'MiErrorPersonalizado'; // Define el nombre del error
5   }
6 }
7
8 // Funcion que lanza la excepcion personalizada
9 function lanzarError() {
10   throw new MiErrorPersonalizado('Ocurrio un error personalizado!');
11 }
12
13 try {
14   lanzarError();
15 } catch (error) {
16   if (error instanceof MiErrorPersonalizado) {
17     console.log('Se atrapo un MiErrorPersonalizado: ' + error.message);
18   } else {
19     console.log('Se atrapo un error desconocido: ' + error.message);
20   }
21 }
```

try: El bloque de código donde puede ocurrir una excepción.

catch: Captura y maneja la excepción. Puede tener un único bloque catch que maneja todas las excepciones.

finally: Se ejecuta siempre, ocurra o no una excepción, ideal para la limpieza de recursos.

```
1 function divide(a, b) {
2   try {
3     if (typeof a !== 'number' || typeof b !== 'number') {
4       throw new TypeError('Ambos argumentos deben ser numeros');
5     }
6     let result = a / b;
7     if (b === 0) {
8       throw new Error('Division por cero');
9     }
10    console.log('Division satisfactoria');
11    return result;
12  } catch (e) {
13    console.log(`Error: ${e.message}`);
14    return null;
15  } finally {
16    console.log('Ejecucion completada');
17  }
18 }
19
20 console.log(divide(10, 2));
21 console.log(divide(10, 0));
22 console.log(divide(10, 'a'));
```

En este ejemplo de JavaScript los casos son similares a los vistos anteriormente, salvo por la gran diferencia de que se utilizan 'ifs' para detectar los respectivos errores y que no hace falta un catch por cada excepción, sino que las excepciones se levantan a través del 'throw new <TipoDeError>', y una vez que se levanta una excepción se termina la ejecución del bloque try y se ejecutará el manejador de excepciones 'catch (e)' que imprimirá "Error: <TipoDeError>", y por último se ejecuta el finally que imprime 'Ejecución completada'.

Conclusión

A lo largo de la carrera se nos enseñan los contenidos de las asignaturas de tal forma que podamos comprender los conceptos fundamentales y otros tantos conceptos necesarios pero de manera superficial. Por ejemplo, el hecho de asignarle un valor a una variable implicaba, desde la postura de un estudiante de primer año, guardar un valor en una posición de memoria. Lo que ocurre en el medio es una incógnita que se resuelve en este tramo de la carrera al aprender, por ejemplo, el concepto del l-valor y el r-valor. Tener conocimiento de los distintos aspectos de las estructuras que usamos en el día a día como programadores nos permite tomar mejores decisiones, entre tantos otros beneficios.

Particularmente en nuestro trabajo nos ha tocado investigar sobre dos de los lenguajes más populares del momento, acá no hay uno mejor que otro, cada uno puede utilizarse para tareas particulares, con las ventajas y desventajas que ofrecen, ambos son excelentes herramientas para poder desarrollar software.

Gracias a este trabajo, nos dimos cuenta lo importante que puede resultar realizar una investigación previa a la hora de la elección de un lenguaje y poder hacer esta elección con fundamentos. También comprendimos la comunidad que hay detrás de cada lenguaje y cómo podemos beneficiarnos entre todos.